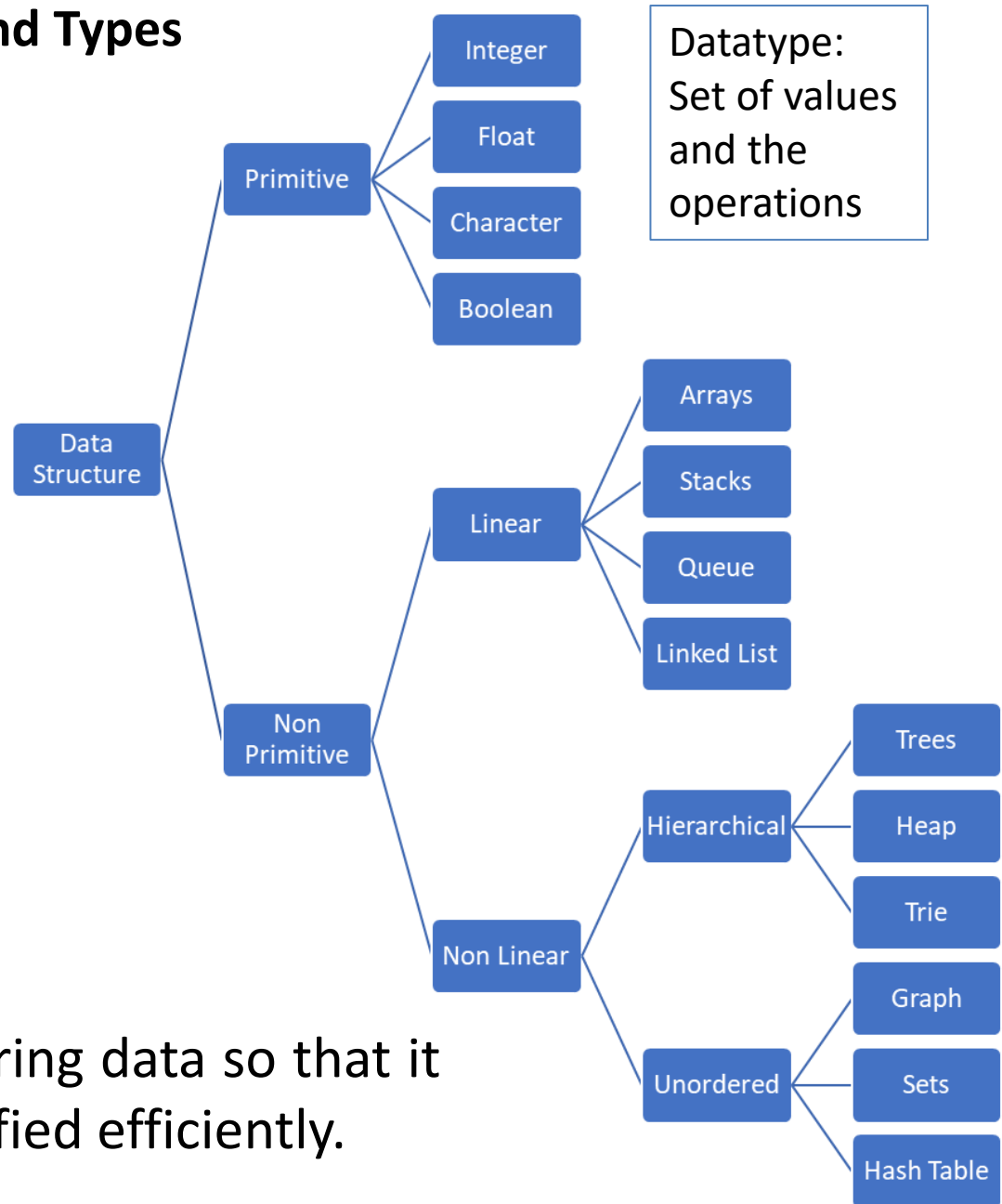


Data Structures- Unit 1

Data Structure: Definition and Types



Way of organizing and storing data so that it can be accessed and modified efficiently.

Real life Examples(Linear Data Structure)

Data elements are arranged sequentially

- **Array**
 - Question Number and corresponding marks
 - Images- Stored in the form of 2D arrays of pixels
 - ...
- **Stack**
 - Undo/Redo
 - Back and forward Button in browser
 - Memory Management
 - ...
- **Queue**
 - Lines in Service Counters
 - Operating system scheduling
 - Customer calls in call centers
- **Linked List**
 - Song in a playlist(node)
 - Browser history
 - Undo(Stack or Linked list?)...

Real life Examples(Non- Linear Hierarchical Data Structure)

Elements are not arranged sequentially and can have multiple connections or relationships

- **Tree**
 - Computers file system(Folders and files)
 - Web page Structure
 - Biological Taxonomy
 - ...
- **Heap**
 - Priority Queueing
 - Sorting
 - ...
- **Trie**
 - Auto completion of words
 - Spell checkers
 - IP routing tables to find the destination IP address
 - ...

Real life Examples(Non- Linear Unordered Data Structure)

- **Graph**

- Social Networks(ex: Facebook)
- Transportation Network
- Web pages(hyperlinks) and search engines
- ...

- **Sets**

- Shopping list(item appears one)
- Contact number in phone
- Players in a team
- ...

- **Hash Table**

- Storing passwords(Secured)
- Image processing(To find image modification)
- ...

Data Type

Collection of values and a set of operations on those values.

- Integer
- Real number
- Character, string etc.,

Structures in C

```
struct structure_name {  
    data_type1 member1;  
    data_type2 member2;  
    ...  
};
```

```
structure_name . member1;  
structure_name . member2;
```

```
structure_ptr -> member1  
structure_ptr -> member2
```

Example- Structures

```
#include<stdio.h>
struct Distance{
    int feet;
    float inch;
};

int main() {
    struct Distance d1, *d2;
    d1.feet=12;
    d1.inch=20.5;
    printf("%d\t%f\n",d1.feet,d1.inch);

    d2->feet=120;
    d2->inch=200;
    printf("%d\t%f",d2->feet,d2->inch);

    d2=&d1;
    printf("\n%d\t%f",d2->feet,d2->inch);
}
```

12	20.500000
120	200.000000
12	20.500000

Example- Pointers

```
#include <stdio.h>
void swap(int x, int y)
{
    int temp = x;
    x = y;
    y = temp;
}
int main()
{
    int a = 40, b = 50;
    printf("Before swap, a = %d and b = %d\n", a, b);
    swap(a, b);
    printf("After swap, a = %d and b = %d\n", a, b);
    return 0;
}
```

Before swap, a = 40 and b = 50

After swap, a = 40 and b = 50

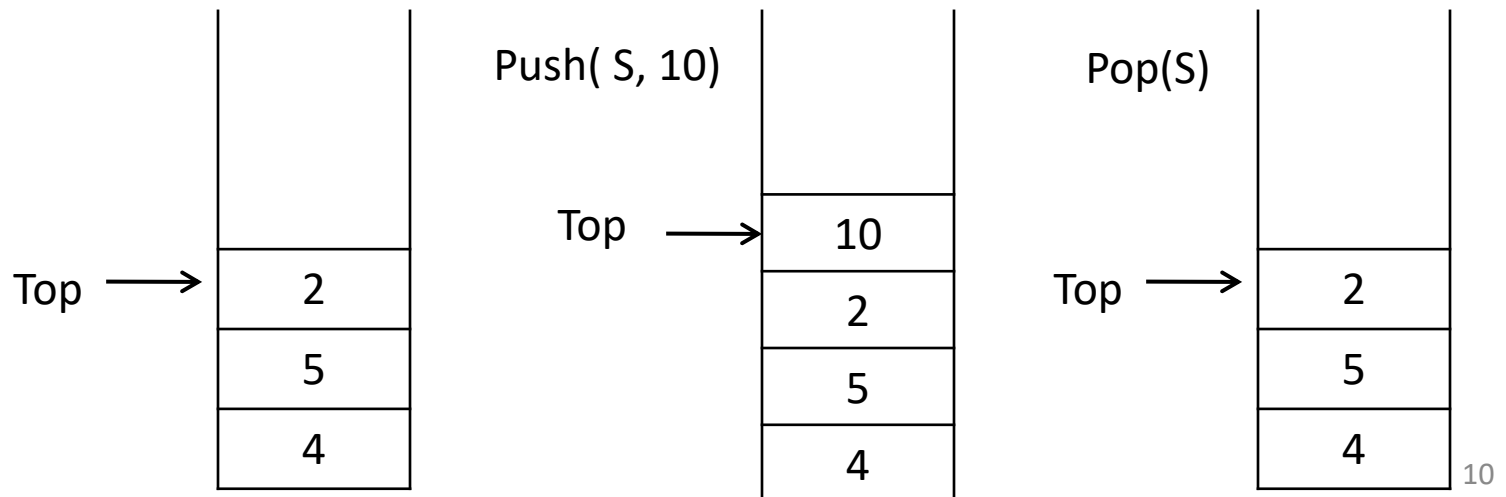
```
#include <stdio.h>
void swap(int *x, int *y)
{
    int temp = *x;
    *x = *y;
    *y = temp;
}
int main()
{
    int a = 40, b = 50;
    printf("Before swap, a = %d and b = %d\n", a, b);
    swap(&a, &b);
    printf("After swap, a = %d and b = %d\n", a, b);
    return 0;
}
```

Before swap, a = 40 and b = 50

After swap, a = 50 and b = 40

Stack

- Definition: An ordered collection of items into which new items may be inserted and deleted at one end called **top** of the stack.
- Primitive Operations
 - Push (S, elt)
 - Pop(S)
- Access Mechanism – LIFO (Last In First Out)



Representing Stacks in C

```
# define MAX_SIZE 10  
  
struct stack {  
    int top;  
    int items[MAX_SIZE];  
};  
  
struct stack s;
```

Possible access to the members
struct stack s;
s.top
s.item[s.top]
(or)

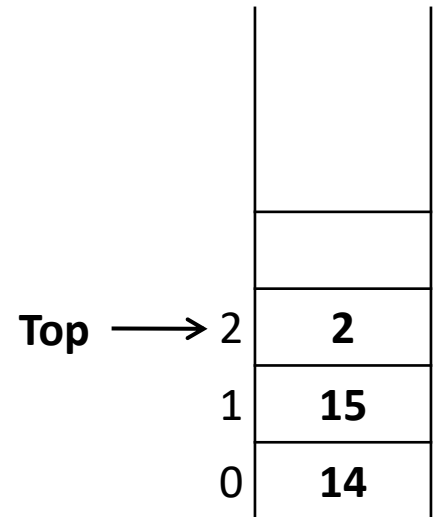
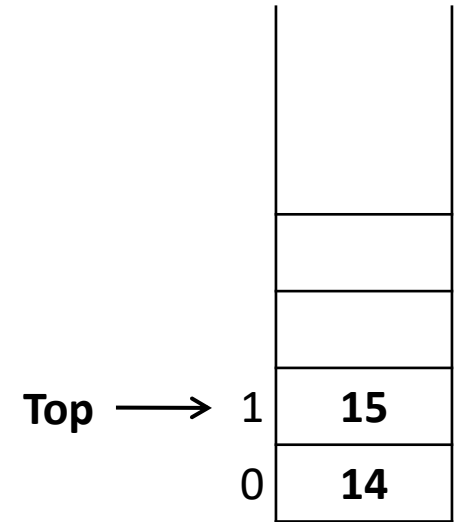
struct stack *s;
s->top
s->items[s->top]

Implementing Push Operation

```
void push(struct stack *ps, int x)
{
    ps -> items[++ps -> top] = x;
    return;
}
```

Implementing Push Operation(Revised)

```
void push(struct stack *ps, int x)
{
    if (ps -> top == STACKSIZE - 1)
    {
        printf("%s", "Stack Overflow");
    }
    else
        ps -> items[++ps -> top] = x;
    return;
}
```



Implementing pop operation

```
int pop(struct stack *ps)
{
    if (isempty(ps))
    {
        printf("%s", "Stack Underflow");
        exit(1);
    }
    return(ps -> items[ps -> top--]);
} /* end pop */
```

```
x = pop (&s);
```

Pop(S)

Top →

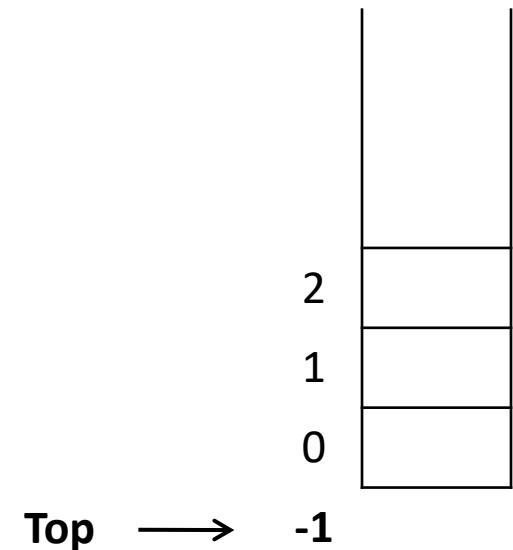
3	5
2	2
1	5
0	4

Top →

2	2
1	5
0	4

Implementing empty operation

```
int isempty(struct stack *ps)
{
    if (ps -> top == -1)
        return (TRUE);
    else
        return (FALSE);
} /* end empty */
```



```
void popandtest(struct stack *ps, int *px, int *pund)
{
    if ( isempty(ps) )
    {
        *pund = TRUE;
        return;
    }
    *pund = FALSE;
    *px = ps -> items[ps -> top--];
    return;
} / * end pop */
```


Implementing Stacktop

```
int stacktop(struct stack *ps)
{
    if ( empty(ps))
    {
        printf("%s", "Stack Underflow");
        exit(1);
    }
    else
        return(ps ->items[ps->top]);
}
```

```
// C Program to Implement a Stack
#include <stdio.h>
#include<stdlib.h>
#define MAX_SIZE 10
#define TRUE 1
#define FALSE 0
```

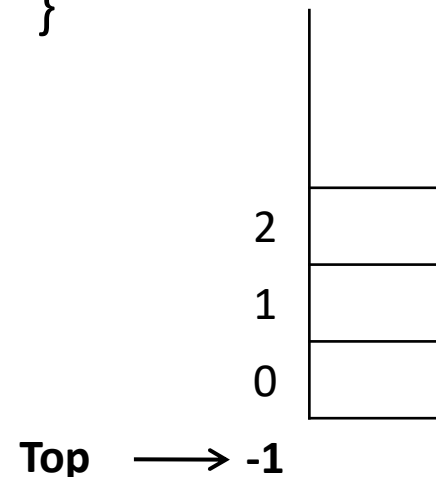
```
// Define a structure for the stack
```

```
struct stack {
    int top;
    int items[MAX_SIZE];
};
```

```
void initialize(struct stack *ps) {
    ps->top = -1;
}
```

```
int isempty(struct stack *ps)
{
    if (ps -> top == -1)
        return (TRUE);
    else
        return (FALSE);
}
```

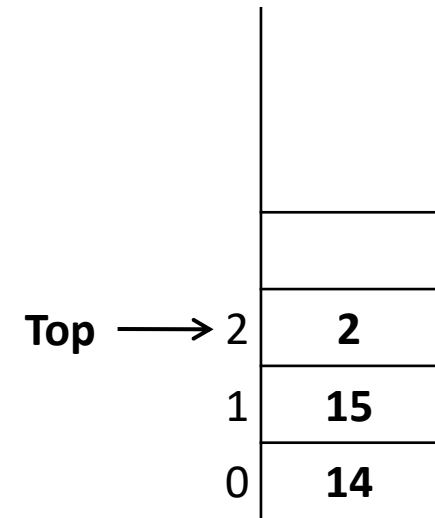
```
int isFull(struct stack *ps)
{
    return ps->top == MAX_SIZE - 1;
}
```



```

void push(struct stack *ps, int x)
{
    if (isFull(ps))
    {
        printf("%s", "Stack Overflow");
    }
    else
    {
        ps -> items[++ps -> top] = x;
        return;
    }
}

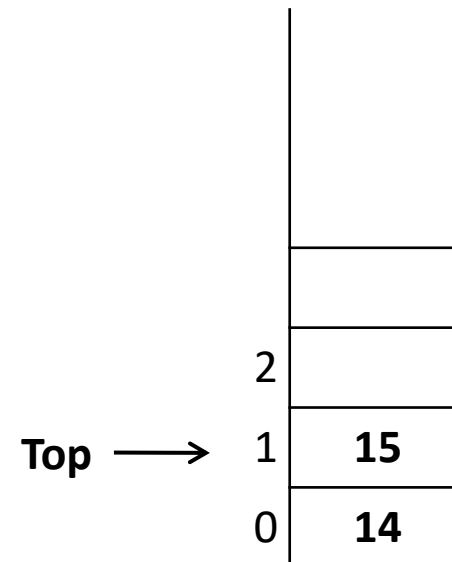
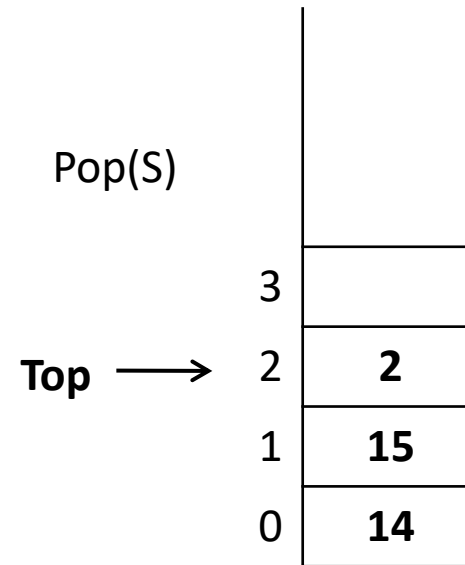
```



```

int pop(struct stack *ps)
{
    if (isempty(ps))
    {
        printf("%s", "Stack
Underflow");
        exit(1);
    }
    return(ps -> items[ps -
>top--]);
}

```



```
int main()
{
    struct stack s;
    initialize(&s);
    push(&s, 14);
    push(&s, 15);
    push(&s, 2);
    while (!isempty(&s))
    {
        printf("Popped element: %d\n", pop(&s));
    }
    return 0;
}
```

Heterogeneous Stack

```
# define STACKSIZE 100
```

```
# define INT 1
```

```
# define FLOAT 2
```

```
# define STRING 3
```

```
struct stackelement {  
    int etype;  
    union {  
        int ival;  
        float fval;  
        char *pval;  
    } element;  
};
```

```
struct stack {  
    int top;  
    struct stackelement items[STACKSIZE];  
};
```

```
struct stack s;  
struct stackelement se;  
  
. . .  
se = s.items [s.top];  
switch (se.etype)  
{  
    case    INTGR : printf("%d\n", se.ival);  
    case      FLT : printf("%f\n", se.fval);  
    case STRING : printf("%s\n", se.pval);  
} /* end switch */
```

Infix, Prefix and Postfix expressions

Examples

$A + B$ (Infix)

$+ A B$ (Prefix)

$A B +$ (Postfix)

Conversion from Infix to Postfix

INFIX	POSTFIX
$A + B - C$	$A B + C -$
$(A + B) \times (C - D)$	$A B + C D - \times$
$A \$ B \times C - D + E / F / (G + H)$	$A B \$ C \times D - E F / G H + / +$
$((A + B) \times C - (D - E)) \$ (F + G)$	$A B + C \times D E - - F G + \$$
$A - B / (C \times D \$ E)$	$A B C D E \$ \times / -$

Conversion from Infix to Prefix

INFIX	PREFIX
$A + B - C$	$- + A B C$
$(A + B) \times (C - D)$	$\times + A B - C D$
$A \$ B \times C - D + E / F / (G + H)$	$+ - \times \$ A B C D / / E F + G H$
$((A + B) \times C - (D - E)) \$ (F + G)$	$\$ - \times + A B C - D E + F G$
$A - B / (C \times D \$ E)$	$- A / B \times C \$ D E$

Evaluating a Postfix Expression

opndstk = the empty stack;

/* scan the input string reading one element at a time into symb */

while (not end of input)

{

 symb = next input character;

if (symb is an operand)

 push(opndstk, symb);

else {

 /*symb is an operator */

 opnd 2 = pop(opndstk);

 opnd 1 = pop(opndstk);

 value = result of applying symb to opnd1 and opnd2;

 push(opndstk, value);

 } /* end else */

}/**end while*/

return(pop(opndstk));

Evaluating the Postfix Expression

6 2 3 + - 3 8 2 / + × 2 \$ 3 +

Postfix Evaluation – Example

6 2 3 + - 3 8 2 / + × 2 \$ 3 +

Symb	opnd 1	opnd 2	value	opndstk
6				6
2				6, 2
3				6, 2, 3
+	2	3	5	6, 5
-	6	5	1	1
3	6	5	1	1, 3
8	6	5	1	1, 3, 8
2	6	5	1	1, 3, 8, 2
/	8	2	4	1, 3, 4
+	3	4	7	1, 7
×	1	7	7	7
2	1	7	7	7, 2
\$	7	2	49	49
3	7	2	49	49, 3
+	49	3	52	52

$$10 \ 5 \ + \ 60 \ 6 \ / \ * \ 8 \ -$$

10

10

5
10

5

15

+

60
15

60

6
60
15

6

10
15

/

150

*

8
150

8

142

-

Evaluate the following Postfix Expression: **7 5 3 2 ^ * 9 2 2 ^ - / + 6 4 * +**

Symb	Operand1	Operand2	Value	Operand Stack
7				7
5				7, 5
3				7, 5, 3
2				7, 5, 3, 2
^	3	2	9	7, 5, 9
*	5	9	45	7, 45
9				7, 45, 9
2				7, 45, 9, 2
2				7, 45, 9, 2, 2
^	2	2	4	7, 45, 9, 4
-	9	4	5	7, 45, 5
/	45	5	9	7, 9
+	7	9	16	16
6				16, 6
4				16, 6, 4
*	6	4	24	16, 24
+	16	24	40	40 (Result)

Evaluate the following Postfix Expression: **4 2 \$ 3 * 3 - 8 4 / 1 1 + / +**

Symb	Operand1	Operand2	Value	Operand Stack
4				4
2				4, 2
\$	4	2	16	16
3				16, 3
*	16	3	48	48
3				48, 3
-	48	3	45	45
8				45, 8
4				45, 8, 4
/	8	4	2	45, 2
1				45, 2, 1
1				45, 2, 1, 1
+	1	1	2	45, 2, 2
/	2	1	1	45, 1
+	45	1	46	46 (Result)

Evaluating a Postfix Expression

```
#include<stdio.h>
#include<stdlib.h>
#include<math.h>
#include<ctype.h>
#define MAXCOLS 80
#define TRUE 1
#define FALSE 0
```

```
struct stack
{
float data[MAXCOLS];
int top;
};
```

```
void push(struct stack *ps, int value)
{
    if (ps->top < MAXCOLS - 1)
    {
        ps->data[++ps->top] = value;
    }
}
```

```
float pop(struct stack *ps)
{
    if (ps->top >= 0)
    {
        return ps->data[ps->top--];
    }
    return 0;
}
```

```
double oper(int symb, double op1, double op2)
{
    switch(symb)
    {
        case '+': return(op1 + op2);
        case '-': return(op1 - op2);
        case '*': return(op1 * op2);
        case '/': return(op1 / op2);
        case '$': return(pow(op1, op2));
        default : printf("%s", "Illegal Operation");
                  exit(1);
    }
}
```

```

double eval(char expr[ ])
{
    int c, position;
    double opnd1,opnd2,value;
    struct stack opndstk;
    opndstk.top = -1;
    for(position =0; (c = expr[position]) !='\0'; position++)
        if (isdigit(c))
            push(&opndstk, (double)(c - '0'));
        else
        {
            opnd2 = pop(&opndstk);
            opnd1= pop(&opndstk);
            value = oper(c,opnd1,opnd2);
            push(&opndstk,value);
        }
    return (pop(&opndstk));
}

```

```
void main( )
{
    char expr[MAXCOLS];
    int  position =0;
    printf("Enter the postfix expression ");
    while((expr[position++] = getchar( )) != '\n');
    expr[--position] = '\0 ';
    printf("%s%s", "The original postfix expression is ", expr, "\n");
    printf("\n%f", eval(expr));
} /* end main */
```

```

double eval(char expr[ ])    // Generic expression
{
    int c, position,newval;
    double opnd1,opnd2,value;
    struct stack opndstk;
    opndstk.top = -1;
    for(position =0; (c = expr[position]) !='\0'; position++)
        if (isdigit(c)) {
            push(&opndstk, (double)(c - '0'));
        }
        else if (isalpha(c))
        {
            printf("Enter the value of %c ",c);
            scanf("%d",&newval);
            push(&opndstk, (double)(newval));
        }
        else {
            opnd2 = pop(&opndstk);
            opnd1= pop(&opndstk);
            value = oper(c,opnd1,opnd2);
            push(&opndstk,value);
        }

        return (pop(&opndstk));
}

```

Evaluation of Prefix expression

- The algorithm for evaluating a prefix expression is as follows:
- Accept a prefix string from the user.
- say $(- * + ABCD)$, let $A=4$, $B=3$, $C=2$, $D=5$
- i.e. $(- * + 4325)$ is the input prefix string.
- Start scanning the string from the right one character at a time.
- If it is an operand, push it in stack.
- If it is an operator, pop opnd1, opnd2 and perform the operation, specified by the operator. Push the result in the stack.
- Repeat these steps until input prefix strings ends.

Evaluation of Prefix expression

opndstk = the empty stack;

Read a prefix string from the user

Reverse the prefix string

while (not end of input)

{

 symb = next input character;

if (symb is an operand)

 push(opndstk, symb);

else {

 /*symb is an operator */

 opnd 1 = pop(opndstk);

 opnd 2 = pop(opndstk);

 value = result of applying symb to opnd1 and opnd2;

 push(opndstk, value);

 } /* end else */

}/**end while*/

return(pop(opndstk));

Example: $-*+4325$

Symbol	opnd1	opnd2	value	opndstack
5				5
2				5, 2
3				5, 2, 3
4				5, 2, 3, 4
+	4	3	7	5, 2
				5, 2, 7
*	7	2	14	5
				5, 14
-	14	5	9	
				9

result



Prefix :+*+5436

Infix:(((5+4)*3)+6)

Symbol	Opnd 1	Opnd 2	value	Opndstack
6				6
3				6 3
4				6 3 4
5				6 3 4 5
+	5	4	9	6 3 9
*	9	3	27	6 27
+	27	6	33	33(Result)

Program to evaluate Prefix Expression

```
#include<stdio.h>
#include<stdlib.h>
#include<math.h>
#include<ctype.h>
#include <string.h>
#define MAXCOLS 80
#define TRUE 1
#define FALSE 0
```

```
struct stack
{
float data[MAXCOLS];
int top;
};
```

```
void push(struct stack *ps, int value)
{
    if (ps->top < MAXCOLS - 1)
    {
        ps->data[++ps->top] = value;
    }
}
```

```
float pop(struct stack *ps)
{
    if (ps->top >= 0)
    {
        return ps->data[ps->top--];
    }
    return 0;
}
```

```
double oper(int symb, double op1, double op2)
{
    switch(symb)
    {
        case '+': return(op1 + op2);
        case '-': return(op1 - op2);
        case '*': return(op1 * op2);
        case '/': return(op1 / op2);
        case '$': return(pow(op1, op2));
        default : printf("%s", "Illegal Operation");
                  exit(1);
    }
}
```

```

double eval(char expr[ ])
{
    int c, position,newval;
    double opnd1,opnd2,value;
    struct stack opndstk;
    opndstk.top = -1;
    for(position =0; (c = expr[position]) !='\0'; position++)
        if (isdigit(c)) {
            push(&opndstk, (double)(c - '0'));
        }
        else if (isalpha(c))
        {
            printf("Enter the value of %c ",c);
            scanf("%d",&newval);
            push(&opndstk, (double)(newval));
        }
        else {
            opnd1 = pop(&opndstk);
            opnd2= pop(&opndstk);
            value = oper(c,opnd1,opnd2);
            push(&opndstk,value);
        }
        return (pop(&opndstk));
}

```

```

void main( )
{
    char expr[MAXCOLS];
    char temp;
    int position =0,j,k;
    printf("Enter the prefix expression ");
    while((expr[position++] = getchar( )) != '\n');
    expr[--position] = '\0';
    printf("%s%s", "The original prefix expression is ", expr);
    k=position;
    for(position=0,j=k-1;position<=j;position++,j--)
    {
        temp=expr[position];
        expr[position]=expr[j];
        expr[j]=temp;
    }
    printf("\n%s%s", "The reversed prefix expression is ", expr);
    printf("\n");
    printf("\nThe result of Postfix Expression is %f ", eval(expr));
} /* end main */

```

Converting an Expression from Infix to Postfix

opstk = the empty stack;

while (not end of input)

{

 symb = next input character;

 if (symb is an operand)

 add symb to the postfix string

 else

 {

 while (!empty(opstk) && prcd(stacktop(opstk), symb))

 {

 topsymb = pop(opstk);

 add topsymb to the postfix string;

 }/*end while */

 push(opstk, symb);

 }

}

while (!(empty(opstk)))

{

 topsymb = pop(opstk);

 add topsymb to the postfix string;

}

```
9  if(empty(opstk)) || symb != ')')  
    push(opstk,symb);  
else  
    topsymb = pop(opstk);
```

Precedence Rules

```
prcd(' ( ' , op) =FALSE;  
prcd(op, ' ( ' ) =FALSE; //for any operator other than )  
prcd(op, ' ) ' ) =TRUE; //for any operator other than (  
prcd(' ) ' , op) =undefined;
```

Example- Infix to Postfix Conversion

A + B * C

	symb	Postfix string	opstk
1	A	A	
2	+	A	+
3	B	AB	+
4	*	AB	+ *
5	C	ABC	+
6		ABC *	
7		ABC * +	

Example- Infix to Postfix Conversion

$$A * B / C - D + E / G$$

	symb	Postfix string	opstk
1	A	A	
2	*	A	*
3	B	AB	*
4	/	AB*	/
5	C	AB*C	/
6	–	AB*C/	–
7	D	AB*C/D	–
8	+	AB*C/D –	+
9	E	AB*C/D –E	+
10	/	AB*C/D –E	+ /
11	G	AB*C/D –EG/	+
		AB*C/D –EG/ +	

Example- Infix to Postfix Conversion

A / B ^ C - D

Current Symbol	Postfix String	Stack
Initial State	—	Empty
A	A	Empty
/	A	/
B	AB	/
^	AB	/^
C	ABC	/^
—	ABC^	/
—	ABC^/	Empty
—	ABC^/	—
D	ABC^/D	—
NULL	ABC^/D-	—

Convert the following Infix Expression to Postfix Expression

1. $(A + B) * C$

2. $A * (B + C * D) + E$

3. $((A - (B + C)) * D) \div (E + F)$

```

opstk = the empty stack;
while (not end of input)
{
    symb = next input character;
    if (symb is an operand)
        add symb to the postfix string
    else
    {
        while (!empty(opstk) && prcd(stacktop(opstk), symb))
        {
            topsymb = pop(opstk);
            add topsymb to the postfix string;
        }/*end while */
        if(empty(opstk) || symb != '(')
            push(opstk,symb);
        else
            topsymb = pop(opstk);
    }
}
while (!empty(opstk))
{
    topsymb = pop(opstk);
    add topsymb to the postfix string;
}

```

Precedence Rules

```

prcd(' ( ', op) =FALSE;
prcd(op, ' ( ')=FALSE; //for any operator other than (
prcd(op, ' ) ')=TRUE; //for any operator other than (
prcd(' ) ', op) =undefined;

```

Example- Infix to Postfix Conversion

(A + B) * C

	symb	Postfix string	opstk
1	(		(
2	A	A	(
3	+	A	(+
4	B	AB	(+
5	)	AB +	
6	*	AB +	*
7	C	AB + C	*
		AB + C *	

$$A * (B + C * D) + E$$

Symbol	Postfix	Stack
A	A	
*	A	*
(	A	* (
B	A B	* (
+	A B	* (+
C	A B C	* (+
*	A B C	* (+ *
D	A B C D	* (+ *
)	A B C D * +	*
+	A B C D * + *	+
E	A B C D * + * E	+
	A B C D * + * E +	

$((A - (B + C)) * D) \$ (E + F)$

symp	Postfix string	opstk
(		(
(		(
A	A	((
−	A	((−
(	A	((− (
B	AB	((− (
+	AB	((− (+
C	ABC	((− (+
)	ABC +	((−
)	ABC + −	(
*	ABC + −	(*
D	ABC + − D	(*
)	ABC + − D *	
\$	ABC + − D *	\$
(	ABC + − D *	\$(
E	ABC + − D * E	\$(
+	ABC + − D * E	\$(+
F	ABC + − D * E F	\$(+
)	ABC + − D * E F +	\$
	ABC + − D * E F + \$	

Convert $A * (B + C) * D$ to postfix notation.

Symbol	Postfix	Stack
A	A	empty
*	A	*
(	A	(*
B	A B	(*
+	A B	+ (*
C	A B C	+ (*
)	A B C +	*
*	A B C + *	*
D	A B C + * D	*
	A B C + * D *	

Program to convert infix to postfix

```
#include <stdio.h>
#include <stdlib.h>
#define MAXCOLS 80
#define TRUE 1
#define FALSE 0
void postfix(char *, char *);
int isoperand(char);
void popandtest(struct stack *, char *, int *);
int prcd(char);
void push(struct stack *, char);
char pop(struct stack *);
```

```

void popandtest(struct stack *ps, int *px, int *pund)
{
    if ( isempty(ps) )
    {
        *pund = TRUE;
        return;
    }
    *pund = FALSE;
    *px = ps -> items[ps -> top--];
    return;
} / * end pop */

```

```

int prec(char c)
{
    if (c == '$')
        return 3;
    else if (c == '/' || c == '*')
        return 2;
    else if (c == '+' || c == '-')
        return 1;
}

```

```
struct stack {  
    int top;  
    char items[MAXCOLS];  
};
```

```
postfix(char infix[ ], char postr[ ])  
{  
    int position, und;  
    int outpos = 0;  
    char symb;  
    struct stack opstk;  
    opstk.top = -1;
```

```

for(position =0; (symb= infix[position]) !='\0';position ++)
    if (isoperand(symb))
        postr[outpos++] = symb;
    else
    {
        popandtest(&opstk, &topsymb,&und);
        while (!und && (prec(topsymb) > prec(symb) ||
            prec(topsymb) == prec(symb)))
        {
            postr[outpos++] = topsymb;
            popandtest(&opstk,&topsymb,&und);
        }
        if (!und)
            push(&opstk,topsymb);
        if (und || (symb != ')'))
            push(&opstk,symb);
        else
            topsymb = pop(&opstk);
    }
while (!empty(&opstk))
    postr[outpos++] = pop(&opstk);
postr[outpos]='\0';
return;
}/*end postfix*/

```

```
void main( )  
{  
    char infix[MAXCOLS];  
    char postr[MAXCOLS];  
    int pos = 0;  
    while (( infix[pos++] = getchar( )) != '\n');  
    infix[--pos] = '\0';  
    printf(“%s%s”, “The original infix expression is”, infix);  
    postfix(infix, postr);  
    printf(“%s\n”, postr);  
}
```

Conversion from Prefix to Postfix

Conversion from Prefix to Postfix

- Accept a prefix string from the user.
- Start scanning the string from right one character at a time.
- If it is an operand, push it in stack.
- If it is an operator, pop opnd1, opnd2 and concatenate them in the order (opnd1, opnd2, optr) as follows:
 - `strcpy(arr,opnd1);`
`strcat(arr,opnd2);`
`strcat(arr,optr);`
- Push the result in the stack.
- Repeat these steps until arr of input prefix string ends.
- Pop the remaining element of the stack, which is the required Postfix notation equivalent to a given Prefix notation.
- Ex: A \$ B * C - D + F * (G +H)

Example:

(+-435) $\rightarrow$ (43-5+)

Symbol	opnd1	opnd2	value	opndstack
5				5
3				5, 3
4				5, 3, 4
-	4	3	43-	5
				5, 43-
+	43-	5	43-5+	
				43-5+

Equivalent
Postfix notation



Conversion from Prefix to Postfix

INFIX : A \$ B * C - D + F * (G +H)			
PREFIX : + - * \$ A B C D * F + G H REVERSE STRING: H G + F * D C B A \$ * - +			
SYMP	OP1	OP2	STACK
H			H
G			H G
+	G	H	G H +
F			GH+ F
*	F	GH+	FGH+*
D			FGH+* D
C			FGH+* D C
B			FGH+* D C B
A			FGH+* D C B A
\$	A	B	FGH+* D C AB\$
*	AB\$	C	FGH+* D AB\$C*
-	AB\$C*	D	FGH+* AB\$C*D-
+	AB\$C*D-	FGH+*	AB\$C*D-FGH+* +

PREFIX: + - * \$ A B C D * F + G H REVERSE STRING: H G + F * D C B A \$ * - +			
SYMP	OP1	OP2	STACK
H			H
G			H G
+	G	H	G H +
F			GH+ F
*	F	GH+	FGH+*
D			FGH+* D
C			FGH+* D C
B			FGH+* D C B
A			FGH+* D C B A
\$	A	B	FGH+* D C AB\$
*	AB\$	C	FGH+* D AB\$C*
-	AB\$C*	D	FGH+* AB\$C*D-
+	AB\$C*D-	FGH+*	AB\$C*D-FGH+* +

Conversion from Prefix to Postfix

INFIX : $(A + B) * C - D / E + F$

PREFIX : $+ - * + A B C / D E F$

REVERSE STRING: $F E D / C B A + * - +$

SYMP	OP1	OP2	STACK
F			F
E			F E
D			F E D
/	D	E	F DE/
C			F DE/ C
B			F DE/ C B
A			F DE/ C B A
+	A	B	F DE/ C AB+
*	AB+	C	F DE/ AB+C*
-	AB+C*	DE/	F AB+C*DE/-
+	AB+C*DE/-	F	AB+C*DE/-F+

PROGRAM TO CONVERT A PREFIX EXPRESSION TO A POST FIX

```
#include<stdio.h>
#include<string.h>
void push(char item[ ],int *top,char s[25][20])
{
    *top=*top+1;
    strcpy(s[*top],item);
}

void *pop(int *top,char s[25][20])
{
    char *item;
    item = s[*top];
    *top = *top-1;
    return item;
}
```

```

void pre_post(char prefix[],char postfix[])
{
    char s[20][20];
    int top,i;
    char symbol, temp[2];
    char *op1,*op2;

    top=-1;
    strrev(prefix);
    for(i=0;i<strlen(prefix);i++)
    {
        symbol=prefix[i];
        temp[0]=symbol;
        temp[1]='\0';

```

```

switch (symbol)
{
    case '+':
    case '-':
    case '*':
    case '/':
    case '^':
        op1=pop(&top,s);
        op2=pop(&top,s);
        strcpy(postfix,op1);
        strcat(postfix,op2);
        strcat(postfix,temp);
        push(postfix,&top,s);
        break;
    default:
        push(temp,&top,s);
    } /*end of switch */
} /* end of for*/

```

```

}/*end of pre_post*/

```

```
void main()
{
    char prefix[20];
    char postfix[20];
    printf("\\n\\n Enter the prefix expression \\n\\n");
    scanf("%s",prefix);
    pre_post(prefix,postfix);
    printf("\\n\\n The postfix expression is %s \\n\\n",postfix);
}
```

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
#define MAX 20
struct stack
{
    int top;
    char arr[MAX];
};
void push(struct stack *,char);
char pop(struct stack *);
int isdigit(char);
void prefix_to_postfix(char instr[ ],char outstr[ ]);
```

```

void main()
{
    char str[MAX],resstr[MAX],temp[MAX];
    int i,z;
    clrscr();
    printf("\\nEnter a prefix string");
    gets(str);
    for(i=0;str[i]!='\\0';i++);
    for(i=i-1,z=0; i>=0 ;z++,i-)
    temp[z] = str[i];
    temp[z] = '\\0';
    printf("\\nPrefix string: %s",str);
    prefix_to_postfix(temp,resstr);
    printf("\\nPostfix string: %s",resstr);
    getch();
}

```

```

void push(struct stack *s,char ch)
{
    if (s->top==MAX-1)
    {
        printf("Stack Overflow!");
        exit(1);
    }
    s->arr[++(s->top)]=ch;
}

char pop(struct stack *s)
{
    if(s->top== -1)
    {
        printf("Stack Underflow!");
        exit(1);
    }

    return(s->arr[(s->top)--]);
}

int isdigit(char ch)
{
    return(ch>='0' && ch<='9');
}

```



```

void prefix_to_postfix(char instr[],char outstr[])
{
    int i,j,ct = 1,z = 0;
    char ch,opnd1,opnd2;
    struct stack stk;
    stk.top=-1;
for(i=0;instr[i]!='\0';i++)
{
    ch=instr[i];
    if(isdigit(ch))
    {
        push(&stk,ch);
    }

```

```

else
{
    if (ct==1)
    {
        opnd1=pop(&stk);
        opnd2=pop(&stk);
        outstr[z++]=opnd1;
        outstr[z++]=opnd2;
        outstr[z++]=ch;
        ct++;
    }
    else
    {
        opnd2=pop(&stk);
        outstr[z++]=opnd2;
        outstr[z++]=ch;
    }
    }/*end else*/
}/*end for */
    outstr[z]='\0';
}

```